

Smart URL Shortener Platform — Business Requirement Document (BRD)

Version: 1.0
Date: April 2026
Tech Stack: Java Spring Boot + ReactJS + PostgreSQL + Redis
Status: Production-Ready Architecture

1. Project Overview

1.1 Project Name

Smart URL Shortener Platform

1.2 Objective

Build a scalable, secure, and high-performance URL shortening platform where users can convert long URLs into short, shareable links. The system provides comprehensive tracking, analytics, user authentication, link lifecycle management, and secure redirection.

1.3 Business Problem

Long URLs are difficult to share, remember, and track. Businesses, marketers, developers, and everyday users need a reliable system to:

- Shorten long URLs into compact, shareable links
- Track total and unique clicks with detailed analytics
- Manage created links (update, delete, enable/disable)
- Set expiry dates for time-sensitive campaigns
- Secure links with ownership and access control
- Detect and block malicious or spam URLs

1.4 Target Users

Actor	Description
Guest	Can shorten URLs without login with limited features (rate-limited)
Registered User	Full access to create, update, delete, and track their own URLs
Admin	Full system control —manage users, URLs, blocked links, and platform analytics

2. Core Modules

2.1 Authentication & User Management

Functional Requirements

- User registration with email uniqueness validation
- Secure login with JWT-based authentication (Access + Refresh tokens)
- Password encryption using BCrypt
- Role-based access control: USER, ADMIN
- Account status control: ACTIVE, BLOCKED, DELETED
- Email verification support
- Brute-force protection via failed login attempt tracking
- Multi-device session management with secure logout
- Soft delete support for user accounts

APIs

Method	Endpoint	Description
POST	/api/v1/auth/register	Register new user
POST	/api/v1/auth/login	Authenticate and receive tokens
POST	/api/v1/auth/refresh	Refresh access token
POST	/api/v1/auth/logout	Invalidate session

2.2 URL Management

Features

- Create short URL from any valid long URL
- Generate random short code using **Base62 Encoding** (0-9, a-z, A-Z)
- Support custom aliases (unique, non-reusable)
- Prevent duplicate aliases across the platform
- Validate URL format and block malicious URLs
- Set optional expiry date (auto-disable via scheduled jobs)
- Enable/disable URL status
- Ownership validation (users manage only their own URLs)
- Bulk URL upload support (CSV)
- Optional password protection for links
- QR code generation for short URLs

APIs

Method	Endpoint	Description
POST	/api/v1/urls	Create short URL
GET	/api/v1/urls	List user's URLs (paginated, filterable)
GET	/api/v1/urls/{id}	Get URL details

Table 3 – continued

Method	Endpoint	Description
PUT	/api/v1/urls/{id}	Update URL (alias, expiry, status)
DELETE	/api/v1/urls/{id}	Delete URL (soft delete)

Sample Request

```
{
  "originalUrl": "https://example.com/products/spring-boot-course",
  "customAlias": "spring-course",
  "expiryDate": "2026-12-31T23:59:59",
  "title": "Spring Boot Course"
}
```

Sample Response

```
{
  "status": 201,
  "message": "Short URL created successfully",
  "data": {
    "id": 1,
    "originalUrl": "https://example.com/products/spring-boot-course",
    "shortCode": "spring-course",
    "shortUrl": "https://shortly.com/spring-course",
    "expiryDate": "2026-12-31T23:59:59",
    "active": true,
    "qrCodeUrl": "https://shortly.com/qr/spring-course"
  }
}
```

2.3 Redirection Engine (Performance Critical)

Requirements

- **O(1) lookup** using short_code database index
- **Redis caching** for ultra-fast resolution under high traffic
- Graceful handling of expired, disabled, or non-existent URLs
- Every valid redirect records click analytics asynchronously

Business Flow

```
Request → Redis Check → DB Fallback (if cache miss) → Record Analytics → Redirect
```

API

Method	Endpoint	Description
GET	/ {shortCode}	Redirect to original URL

Business Rules

Condition	Response
Short code does not exist	HTTP 404 Not Found
URL expired	Expired page (HTTP 410 Gone)
URL disabled	Inactive page (HTTP 403 Forbidden)
URL password-protected	Password prompt page
Valid URL	HTTP 302 Found Redirect

2.4 Analytics Engine

Tracked Data per Click

- IP Address
- Browser (Chrome, Firefox, Edge, Safari, etc.)
- Operating System (Windows, macOS, Linux, Android, iOS)
- Device Type (Desktop, Mobile, Tablet)
- Geo Location (Country, City) via IP geolocation
- Timestamp
- Referrer URL

APIs

Method	Endpoint	Description
GET	/api/v1/analytics/{urlId}	Get analytics for a specific URL
GET	/api/v1/analytics/dashboard	Get user dashboard summary

Analytics Response

```
{
  "totalClicks": 1245,
  "uniqueClicks": 895,
  "topBrowsers": ["Chrome", "Edge", "Firefox"],
  "topDevices": ["Desktop", "Mobile"],
  "topCountries": ["USA", "India", "UK"],
  "dailyClicks": [
    { "date": "2026-04-20", "clicks": 120 },
    { "date": "2026-04-21", "clicks": 145 },
    { "date": "2026-04-22", "clicks": 98 }
  ],
  "referrers": [
    { "source": "twitter.com", "clicks": 450 },
    { "source": "facebook.com", "clicks": 320 }
```

```
1
}
```

2.5 User Dashboard Module

Features

- View total links created
- View total clicks across all links
- View active vs. expired links count
- Recent links list
- Search and filter URLs by status, date, or keyword
- Copy short URL to clipboard
- Edit or delete links
- View detailed analytics per link

Frontend Screens

- Landing Page
- Login / Register
- Dashboard (overview)
- My URLs (list with search/filter)
- Create Short URL
- URL Detail + Analytics
- User Profile

2.6 Admin Panel

Features

- View all registered users with status
- View all URLs across the platform
- Disable/enable harmful or spam URLs
- Block/unblock user accounts
- View platform-wide analytics
- Detect suspicious links automatically

APIs

Method	Endpoint	Description
GET	/api/v1/admin/users	List all users
PUT	/api/v1/admin/users/{id}/ block	Block/unblock user
GET	/api/v1/admin/urls	List all URLs
PUT	/api/v1/admin/urls/{id}/ disable	Disable/enable URL
DELETE	/api/v1/admin/urls/{id}	Hard delete spam URL

Table 7 – continued

Method	Endpoint	Description
GET	/api/v1/admin/dashboard	Platform analytics summary

3. Database Design

3.1 Users Table (Production-Level)

```
CREATE TABLE users (  
  id BIGSERIAL PRIMARY KEY,  
  
  -- Identity  
  username VARCHAR(50) UNIQUE NOT NULL,  
  email VARCHAR(150) UNIQUE NOT NULL,  
  password_hash VARCHAR(255) NOT NULL,  
  
  -- Profile  
  full_name VARCHAR(150),  
  phone_number VARCHAR(20),  
  
  -- Role & Access  
  role VARCHAR(20) NOT NULL DEFAULT 'USER',  
  status VARCHAR(20) NOT NULL DEFAULT 'ACTIVE',  
  
  -- Security  
  email_verified BOOLEAN DEFAULT FALSE,  
  failed_login_attempts INT DEFAULT 0,  
  last_login_at TIMESTAMP,  
  
  -- Audit  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  -- Soft Delete  
  deleted BOOLEAN DEFAULT FALSE  
);
```

Key Design Decisions:

- failed_login_attempts — brute-force protection
- email_verified — real-world auth system requirement
- deleted — soft delete instead of hard delete for data integrity
- status — control access without permanently deleting user

3.2 User Sessions Table

```
CREATE TABLE user_sessions (  
  id BIGSERIAL PRIMARY KEY,
```

```

    user_id          BIGINT REFERENCES users(id),
    refresh_token    TEXT NOT NULL,
    device_info      VARCHAR(255),
    ip_address       VARCHAR(100),
    expires_at       TIMESTAMP,
    created_at       TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

Purpose: Manage multiple devices, secure logout, refresh token tracking

3.3 URLs Table

```

CREATE TABLE urls (
    id                BIGSERIAL PRIMARY KEY,
    user_id           BIGINT REFERENCES users(id),

    original_url      TEXT NOT NULL,
    short_code        VARCHAR(50) UNIQUE NOT NULL,
    title             VARCHAR(150),

    active            BOOLEAN DEFAULT TRUE,
    expiry_date       TIMESTAMP,
    password_hash     VARCHAR(255), -- optional link password

    total_clicks      BIGINT DEFAULT 0,
    unique_clicks     BIGINT DEFAULT 0,

    created_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE INDEX idx_short_code ON urls(short_code);
CREATE INDEX idx_user_id ON urls(user_id);
CREATE INDEX idx_expiry_date ON urls(expiry_date);

```

3.4 URL Clicks Table

```

CREATE TABLE url_clicks (
    id                BIGSERIAL PRIMARY KEY,
    url_id            BIGINT NOT NULL REFERENCES urls(id),

    ip_address        VARCHAR(100),
    browser           VARCHAR(100),
    os                VARCHAR(100),
    device_type       VARCHAR(50),

    referrer          TEXT,
    country            VARCHAR(100),
    city              VARCHAR(100),

    clicked_at        TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

```

```
);  
  
CREATE INDEX idx_url_id ON url_clicks(url_id);  
CREATE INDEX idx_clicked_at ON url_clicks(clicked_at);
```

4. Technology Stack

4.1 Backend

Technology	Purpose
Java 21	Programming language
Spring Boot	Application framework
Spring Security	Authentication & authorization
Spring Data JPA	Database access
JWT	Token-based authentication
PostgreSQL	Primary database
Redis	Caching & session store
Flyway	Database migrations
Swagger/OpenAPI	API documentation
Docker	Containerization

4.2 Frontend

Technology	Purpose
ReactJS	UI framework
Vite	Build tool
React Router	Client-side routing
Axios	HTTP client
React Query	Server state management & caching
Tailwind CSS	Styling
Recharts	Analytics charts
React Hook Form	Form handling

5. Backend Architecture

5.1 Recommended Package Structure

```
com.project.urlshortener  
├─ auth  
├─ user  
├─ url  
├─ analytics  
├─ admin  
└─ security
```



```
├─ exception
├─ config
├─ common
└─ scheduler
```

5.2 Key Backend Patterns

- **DTO-based** request/response models
- **Global exception handling** (clear distinction between 400 and 500 errors)
- **JWT authentication filter** with access + refresh tokens
- **Password hashing** with BCrypt
- **URL validation** using regex + malicious URL detection
- **Base62 encoding** for short code generation
- **Redis caching** for short-code lookup optimization
- **Pagination and filtering** for all list endpoints
- **@Scheduled jobs** for expired link cleanup and analytics aggregation
- **Clean Architecture:** Controller → Service → Repository → DTO

5.3 Short Code Generation Strategy

Avoid basic random strings. Use:

1. Generate unique sequential ID from database
2. Encode ID using **Base62** (characters: 0-9, a-z, A-Z)
3. Result: guaranteed uniqueness + shorter length

5.4 Redis Caching Flow

```
GET /{shortCode}
→ Check Redis for original_url
→ If HIT → redirect immediately
→ If MISS → query PostgreSQL → store in Redis → redirect
```

5.5 Rate Limiting

- Guest users: max **5 URLs/day** (IP-based)
- Registered users: higher limits based on role
- Prevent abuse and DDoS on redirection endpoint

6. Security Requirements

Requirement	Implementation
Authentication	JWT (access + refresh tokens)
Password Storage	BCrypt hashing
Authorization	Role-based access control (RBAC)
Input Validation	DTO-level + regex validation

Table 10 – continued

Requirement	Implementation
URL Safety	Malicious URL detection, XSS/phishing prevention
Rate Limiting	IP-based for guests, user-based for registered
CORS	Strict configuration
Admin Protection	Admin-only endpoints secured
Session Security	Refresh token rotation, device tracking
Brute Force	Failed login attempt tracking & account lockout

7. Frontend Pages

7.1 Landing Page

- Project introduction & branding
- URL input box with shorten button
- Recent public links (optional)
- Login / Register buttons

7.2 Authentication Pages

- **Login:** Email + Password
- **Register:** Full name + Email + Password + Confirm password
- Email verification flow

7.3 Dashboard Page

- Total URLs card
- Total clicks card
- Active URLs count
- Expired URLs count
- Recent links table
- Quick shorten widget

7.4 My URLs Page

- Paginated URL list
- Search by title or short code
- Filter by status (active, expired, disabled)
- Copy short URL button
- Edit / Delete actions
- View analytics link

7.5 Analytics Page

- Total clicks & unique clicks
- Daily clicks line chart
- Browser distribution pie chart

- Device type bar chart
- Top referrers table
- Geo-location map (optional)

7.6 Admin Dashboard

- Total users count
 - Total URLs count
 - Total clicks count
 - Suspicious URLs alert
 - User management table
 - URL management table
 - Platform activity charts
-

8. Business Rules

1. Short code must be **globally unique**
 2. Custom alias **cannot be reused** even after deletion
 3. Expired URLs **must not redirect**
 4. Disabled URLs **must not redirect**
 5. Guest users can create **limited URLs** (rate-limited)
 6. Registered users can manage **only their own URLs**
 7. Admin can manage **all URLs and users**
 8. Every valid redirect **must create a click record**
 9. Invalid/malicious URLs **must be rejected at creation**
 10. Passwords **must be stored in encrypted format only**
 11. Soft-deleted users **must not be able to log in**
 12. Blocked users **must have all URLs disabled automatically**
-

9. Development Phases

Phase 1 — Foundation

- Initialize Spring Boot + React projects
- Design database schema with Flyway migrations
- Implement URL entity and Base62 short code generation
- Build basic redirect endpoint
- Create landing page with shorten form

Phase 2 — Authentication

- User registration & login
- JWT security (access + refresh tokens)
- User session management
- Role-based access control
- Protected routes in React

Phase 3 — URL Management

- CRUD operations for URLs
- Custom alias support
- Expiry date handling
- User dashboard & My URLs page
- Search, filter, and pagination

Phase 4 — Analytics

- Click tracking (IP, browser, OS, device, geo, referrer)
- Analytics aggregation API
- Charts in React (Recharts)
- Dashboard analytics summary

Phase 5 — Performance & Security

- Redis caching for redirection
- Rate limiting implementation
- URL validation & malicious detection
- Background jobs (@Scheduled)

Phase 6 — Admin & Advanced Features

- Admin panel
- User/URL management
- QR code generation
- Link password protection
- Bulk URL upload (CSV)

Phase 7 — Production

- Dockerize backend & frontend
- Nginx reverse proxy
- HTTPS/SSL configuration
- Swagger API documentation
- Deploy to cloud (AWS/GCP/Azure)
- CI/CD pipeline setup

10. Advanced / Production Features

10.1 High-Impact Add-ons

Feature	Impact
QR Code Generation	Shareable offline links
Custom Domain Support	White-label solution
Link Password Protection	Enhanced privacy
Bulk URL Upload (CSV)	Power user feature
Email Verification	Real-world auth standard

Table 11 – continued

Feature	Impact
Background Jobs	Expired cleanup, analytics aggregation

10.2 Observability

- **Logging:** Logback structured logging
- **Monitoring:** Spring Boot Actuator + metrics
- **Error Tracking:** Centralized error logging
- **Health Checks:** Database, Redis, external services

10.3 Deployment Architecture

```
Internet → Nginx (SSL) → Spring Boot API
      ↓
    PostgreSQL
      ↓
    Redis Cache
```

11. Resume Highlight

Developed a full-stack URL Shortener platform using Java 21, Spring Boot, ReactJS, PostgreSQL, Redis, and JWT authentication. Implemented secure URL generation with Base62 encoding, custom aliases, expiry-based redirection, comprehensive click analytics (browser, device, geo-location), user dashboard with charts, admin panel for platform management, Redis-based caching for O(1) redirection lookup, rate limiting, QR code generation, and Docker-based cloud deployment.

12. Key Interview Talking Points

This project demonstrates:

- **System Design:** Modular, layered architecture with clear separation of concerns
- **Security:** JWT with refresh tokens, RBAC, BCrypt, brute-force protection, input validation
- **Performance:** Redis caching, database indexing, Base62 encoding, pagination
- **Database Design:** Normalized schema, soft deletes, audit fields, proper indexing
- **Real-World Patterns:** DTOs, global exception handling, scheduled jobs, rate limiting
- **API Design:** RESTful standards, versioning, Swagger documentation
- **Full-Stack Integration:** React Query caching, protected routes, responsive UI
- **DevOps:** Docker containerization, Nginx reverse proxy, CI/CD readiness

13. Final Recommendations

To maximize hiring impact:

1.  **Implement Redis caching** (mandatory for performance)
2.  **Add analytics with charts** (visual impact)
3.  **Use clean architecture** (Controller → Service → Repository → DTO)
4.  **Write proper exception handling** (distinguish 400 vs 500)
5.  **Add Swagger documentation** (professional API presentation)
6.  **Deploy it live** (very important — shows ownership)
7.  **Add Docker + basic CI/CD** (DevOps awareness)
8.  **Include QR codes** (polished user experience)

Document prepared for production-level URL Shortener development.